

# Cuestiones de Protocolos de transporte

C 7.1 (Stallings 20.1) ¿Qué elementos se necesitan para direccionar al usuario destino en el nivel de transporte?

C 7.2 (Stallings 20.10) ¿Cómo puede usarse TCP para lidiar con la congestión de la red o de Internet?

C 7.3 ¿Cuáles son las principales diferencias y similitudes entre TCP y UDP?

C 7.4 ¿Cómo delimita TCP los diferentes mensajes que se envían dentro de un flujo?

C 7.5 ¿Qué primitivas de TCP emplea un servidor que no emplea un cliente?

C 7.6 ¿Qué es un puerto “*well-known*”?

C 7.7 ¿Quién determina cuales son los puertos “*well-known*”?

C 7.8 ¿Para qué vale el *flag* SYN de TCP?

C 7.9 ¿Para qué vale el *flag* FIN de TCP?

C 7.10 ¿Para qué vale el *flag* ACK de TCP?

C 7.11 Razona cómo el checksum de TCP permitiría detectar que se han entregado datos a un destino incorrecto.

C 7.12 ¿En qué situación será necesario emplear la cabecera opcional de escalado de la ventana de TCP?

C 7.13 Razona por qué no se puede emplear el mismo puerto del cliente para reestablecer una conexión TCP justo después de haberla cerrado.

C 7.14 ¿Para qué se emplea el estado TIME\_WAIT de una conexión TCP?

C 7.15 ¿Qué proporciona UDP que no proporcione ya IP?

C 7.16 Indica dos ejemplos de aplicaciones en las que interesa emplear UDP en lugar de TCP y expón el motivo.

# Problemas de Protocolos de transporte

P 7.1 (Tanenbaum 6.23) La fragmentación y el reensamblado de datagramas se manejan a través de IP y son transparentes para TCP. ¿Significa esto que TCP no tiene que preocuparse porque los datos lleguen en el orden equivocado?

P 7.2 (+- Tanenbaum 6.27) La cabecera TCP contiene un campo de *Acknowledgement Number* de 32 bits y un bit de confirmación ACK en la cuarta palabra. ¿Acaso este bit agrega realmente algo? ¿Por qué sí o por qué no?

P 7.3 (Tanenbaum 6.28) La máxima carga útil de un segmento TCP es de 65.495 bytes. ¿Por qué se eligió ese extraño número?

P 7.4 (+- Stallings 19.17) Cuando existen múltiples rutas de igual coste a un destino, un algoritmo de enrutamiento puede distribuir el tráfico de igual forma entre ellas. Esto se llama *balanceo de carga*. ¿Qué efectos tiene este balanceo de carga en un protocolo de la capa de transporte, como TCP?

P 7.5 (Stallings 20.17) Suponiendo que el *round-trip time* (RTT) entre dos equipos es de 100 ms, y que ambos equipos usan una ventana de TCP de 32 Kbytes. ¿Cuál es el *throughput* máximo que se puede obtener en esta situación empleando TCP?

P 7.6 (Stallings 20.18) Suponiendo que dos equipos están conectados entre ellos a través de un enlace de 100 Mbps y que el *round-trip time* (RTT) entre ellos es de 1 ms. ¿Cuál es el tamaño mínimo de la ventana de TCP que permitiría obtener el máximo *throughput* posible entre ellos? Nota: Asume que no hay *overheads*.

## Solución de los problemas

*P 7.1 Un segmento TCP se encapsula en un datagrama IP que es enviado por la red. En base a las redes que atraviere, este datagrama puede fragmentarse o no. En caso de que lo haya sido, el receptor es el encargado de recopilar los diferentes fragmentos y reensamblar el datagrama IP original, del que se des-encapsula el segmento TCP. Sin embargo, diferentes datagramas IP pueden recibirse, ya sea fragmentados o no, en diferente orden, por lo que los segmentos TCP encapsulados en ellos, también se recibirán desordenados. Entonces, TCP debe encargarse de reordenarlos. Luego, son conceptos diferentes, la fragmentación de IP se encarga de reordenar los diferentes fragmentos correspondientes a un mismo datagrama, mientras que TCP se encarga de reordenar los diferentes segmentos que forman parte del mismo flujo.*

*P 7.2 Para determinar si el bit indicado aporta o no debemos plantearnos si el campo del número de confirmación es suficiente para manejar las confirmaciones por sí solo. Primero, hay que tener claro que no todos los segmentos TCP tienen que confirmar datos, por lo que hay que diferenciar entre si se está confirmando o no y hasta dónde se está confirmando. Segundo, ¿podríamos implementar ambos aspectos con un único campo de 32 bits? En TCP se confirma el siguiente byte que se espera recibir (lo que implícitamente confirma hasta el anterior), que puede tomar cualquier valor de 32 bits. Entonces, no podemos reservar un valor especial para indicar que no se está confirmando nada, ya que dicho valor coincide con un valor posible a confirmar del flujo. Luego, Podemos concluir que es imprescindible el uso de al menos un bit adicional al campo acknowledgement number de 32 bits.*

*P 7.3 Hay que pensar en que cada segmento TCP se encapsula en un datagrama IP, que tiene un campo de longitud total de 16 bits. Este permite representar valores entre 0 y  $2^{16}-1=65.535$ . En el campo de longitud total se incluyen 20 octetos de la cabecera IP, lo que nos da 65.515 de máxima carga útil en IP. Finalmente, hay que contar con la cabecera TCP, que también es de como mínimo 20 octetos, por lo que  $65.515-20=65.495$  bytes. Ese justamente es el valor de máxima carga útil de un segmento TCP. Por lo que, resumiendo, se obtiene ese valor de la relación con el tamaño máximo de un datagrama IP.*

*P 7.4 Cuando un algoritmo de enrutamiento realiza balanceo de carga distribuye los paquetes de datos entre las rutas disponibles. Sin embargo, TCP, siendo un protocolo orientado a la conexión, tiene en cuenta el orden de los paquetes y la confiabilidad en la entrega. Si los paquetes de una misma conexión TCP se envían por rutas diferentes, puede haber problemas de reordenamiento de los paquetes en el destino, ya que las rutas pueden tener diferentes latencias. Aunque TCP tiene mecanismos para manejar la reordenación y la retransmisión de paquetes perdidos, el reordenamiento excesivo de los paquetes puede generar un aumento de la latencia, lo que afectaría al rendimiento de la conexión visto desde el punto de vista del aplicativo. Además, si una de las rutas por las que se envía el tráfico experimenta una mayor congestión o falla, TCP también puede*

*reducir considerablemente el rendimiento debido a sus mecanismos de control de flujo. En resumen, el balanceo de carga de un algoritmo de enrutamiento puede afectar al comportamiento de TCP, especialmente en términos de latencia y posible reordenamiento de paquetes, lo que podría conllevar una disminución en el rendimiento de la comunicación.*

*P 7.5 Cada equipo puede enviar 32 Kbytes de datos en los 100 ms del RTT, ya que antes de dicho tiempo es imposible recibir confirmaciones de dichos datos. Según se van recibiendo confirmaciones, se vuelve a abrir la ventana, pero como mucho se podrán enviar otros 32 Kbytes de datos en los siguientes 100 ms por el mismo motivo. Asumiendo que 1 Kbyte son 1.024 bytes, el throughput máximo que puede obtenerse en esta situación es:*

$$V = \frac{32 \times 1024 \times 8 \text{ bits}}{0.1 \text{ segundos}} = 2.621.440 \text{ bps} \cong 2,62 \text{ Mbps}$$

*P 7.6 Para obtener el mayor throughput posible entre los equipos, la ventana TCP tiene que dar cabida a todos los datos que envía uno de los equipos sin pausarse durante el tiempo que transcurre hasta empezar a recibir las confirmaciones. En 1 ms a 100 Mbps se pueden transmitir hasta  $\frac{100 \times 10^6 \times 0,001}{8} = 12.500$  bytes. Luego, este es el tamaño mínimo de la ventana que permite obtener el mayor throughput posible entre los equipos.*